

FAPI 2.0 Security Profile

Complete Flow Diagram & Security Analysis

Protocol:	OAuth 2.1 with FAPI 2.0 Security Profile
Token Binding:	DPoP (Demonstrating Proof-of-Possession)
Authorization:	PAR (Pushed Authorization Requests)
Code Protection:	PKCE (Proof Key for Code Exchange)
Compliance:	FAPI 2.0, Open Banking, Financial Services
Document Date:	November 04, 2025

Table of Contents

- 1. Flow Overview
- 2. Step 1: PAR (Pushed Authorization Request)
- 3. Step 2: Authorization (Browser Flow)
- 4. Step 3: Token Exchange
- 5. Step 4: API Request with DPoP Token
- 6. Step 5: Subsequent API Requests
- 7. Security Stack Summary
- 8. Attack Resistance Matrix
- 9. Key Takeaways

1. Flow Overview

FAPI 2.0 (Financial-grade API) with DPoP provides bank-grade security for OAuth 2.0 flows. This document details each step of the authentication flow and explains the security benefits at each stage.

Step	Component	Purpose
1	PAR (Pushed Authorization)	Secure parameter storage
2	Authorization (Browser)	User authentication & consent
3	Token Exchange	Code → Token with PKCE + DPoP
4	API Request	DPoP-bound token usage
5	Subsequent Requests	Fresh proofs for each call

2. Step 1: PAR (Pushed Authorization Request)

Security Features:

- DPoP proof validation (cryptographic proof of possession)
- Server-side parameter storage (protected from tampering)
- Short-lived request_uri (60 seconds, single-use)
- Parameter integrity protection

What It Protects Against:

Threat	Protection Mechanism
Parameter Injection	Parameters pushed server-side, not in URL
Authorization Tampering	Request stored securely, can't be modified
Phishing	Attacker can't craft malicious authorization URLs
Request Replay	request_uri expires and is single-use
Browser History Leakage	Sensitive params not in URL/browser history
Client Impersonation	DPoP proof validates client has private key

Request Example:

```
POST /realms/fapi-demo/.../par/request
DPoP: eyJhbG... (proof_1)
Body: response_type=code&client_id=...&code_challenge=E9MeIhoa20w...
```

3. Step 2: Authorization (Browser Flow)

Security Features:

- User authentication (login required)
- Explicit user consent (scope transparency)
- Redirect URI validation (whitelist enforcement)
- State parameter (CSRF protection)
- Nonce (replay protection for ID token)

What It Protects Against:

Threat	Protection Mechanism
Authorization Code Intercept	Code useless without PKCE verifier + DPoP key
CSRF Attacks	State parameter prevents cross-site attacks
Phishing	User sees legitimate Keycloak domain
Open Redirect	Strict redirect URI whitelist
Unauthorized Access	User must explicitly consent
Session Fixation	Nonce binds session

Important: No DPoP needed for browser redirect - this step happens in the user's browser.

4. Step 3: Token Exchange

Security Features:

- PKCE validation (code_verifier must hash to code_challenge)
- NEW DPOP proof (different from PAR proof, different jti)
- Authorization code validation (must be valid, not expired, not reused)
- JKT calculation and binding (token permanently bound to key)

PKCE Protection Mechanism:

PKCE (Proof Key for Code Exchange) ensures only the original client can exchange the authorization code. The client sends a **code_verifier** that must hash to the **code_challenge** sent in PAR.

PAR Request	code_challenge = SHA256(code_verifier)
Token Request	code_verifier = original plaintext
Validation	SHA256(code_verifier) == code_challenge ✓

DPoP Token Binding:

Keycloak calculates the JWK thumbprint (JKT) from the DPOP proof's public key and embeds it in the access token's **cnf.jkt** claim. The token is now permanently bound to this key!

Token JWT Payload:

```
{
  "sub": "user123",
  "aud": "api",
  "cnf": {
    "jkt": "-ScGL9f6Qfoyv_UCHl6q55_FMNvLX5f..."
  }
}
```

5. Step 4: API Request with DPoP Token

Security Features:

- DPoP Authorization scheme (Authorization: DPoP)
- Fresh DPoP proof for each request (new jti, current timestamp)
- Access token hash (ath) proves token possession
- JKT matching (token's cnf.jkt must match proof's JKT)
- Request binding (htm + htu bind proof to specific request)

Complete Validation Chain:

Step	Validation
1	JWT signature valid (ES256)
2	Token not expired
3	Extract cnf.jkt from token
4	DPoP proof signature valid
5	htm matches request method
6	htu matches request URL
7	Calculate JKT from proof's JWK
8	Token JKT == Proof JKT ✓
9	ath = SHA256(access_token) ✓
10	jti not seen before (replay check)

Token Theft Protection:

Even if an attacker steals the access token, they CANNOT use it because they don't have the private key to generate valid DPoP proofs. The token's JKT won't match the attacker's key.

6. Attack Resistance Matrix

Comparison of security posture with and without FAPI 2.0 + DPoP:

Attack Type	Without FAPI	With FAPI 2.0 + DPoP
Authorization Code Interception	■ Vulnerable	■ Protected (PKCE)
Token Theft	■ Vulnerable	■ Protected (DPoP)
Token Replay	■ Vulnerable	■ Protected (jti)
Man-in-the-Middle	■ Vulnerable	■ Protected (DPoP+PKCE)
Phishing	■ Vulnerable	■ Protected (PAR)
Parameter Injection	■ Vulnerable	■ Protected (PAR)
CSRF	■■ Partial	■ Protected (State+PAR)
Open Redirect	■ Vulnerable	■ Protected (URI whitelist)
Client Impersonation	■ Vulnerable	■ Protected (DPoP key)
Request Forgery	■ Vulnerable	■ Protected (htm+htu)
Token Export	■ Vulnerable	■ Protected (JKT binding)

7. Security Stack Summary

Defense in Depth - Multiple Layers:

Layer	Component	Protection
1	PAR	Parameter tampering, phishing, injection
2	Authentication	Unauthorized access, CSRF
3	PKCE	Authorization code interception
4	DPoP Binding	Token theft, replay, export
5	DPoP Proofs	Request replay, MITM, stolen proofs

Key Security Properties:

Property	Implementation	Benefit
Sender Constraint	DPoP (token→key)	Stolen tokens useless
Code Protection	PKCE	Intercepted codes useless
Parameter Integrity	PAR	Can't tamper with requests
Proof Freshness	jti + iat	Can't replay old proofs
Request Binding	htm + htu	Can't reuse proofs
Token Possession	ath claim	Must have actual token
User Consent	Explicit auth	User controls access

8. Key Takeaways

Why This Flow is Secure:

- **Multiple Independent Protections:** Each layer protects against different attacks. Even if one layer is bypassed, others remain.
- **Cryptographic Binding:** Tokens are bound to keys using JKT, not just bearer tokens that anyone can use.
- **Request-Specific Validation:** Each request is independently validated with fresh proofs. No trust based on previous requests.
- **Standards-Based:** Fully compliant with FAPI 2.0, OAuth 2.1, and DPoP RFC 9449. Industry best practices.
- **Proven Security:** Used by banks and financial institutions worldwide for Open Banking and financial APIs.

Real-World Protection Example:

Scenario: User on public WiFi at a coffee shop

Without FAPI	With FAPI 2.0 + DPoP
Man-in-the-middle steals token → Attacker uses token → ■ Account compromised	Man-in-the-middle steals token → Can't generate DPoP proof (no key) → ■ Account protected!

Conclusion:

FAPI 2.0 with DPoP provides **bank-grade security** through defense in depth, cryptographic binding, and proof of possession. Your implementation is **secure, compliant, and production-ready** for financial services, Open Banking, and other high-security applications.

■ FAPI 2.0 Compliant	■ OAuth 2.1 Standards
■ DPoP Token Binding	■ PAR Required

■ PKCE Protection	■ ES256 Signatures
■ Bank-Grade Security	■ Production Ready